AI Conversation Log— Load Testing Session

Context: Pulses pipeline load testing to achieve 1000 ev/s sustained throughput

Introduction

This document records the key decision points during the load testing session where AI-assisted analysis and human judgment diverged. These moments of productive disagreement were critical to solving the problem efficiently — not by blindly following AI suggestions, but by rigorously evaluating them against evidence before accepting or rejecting them.

Moment 1: Rejecting Aggregation Window Reduction (200ms)

AI's suggestion: "Reducing the aggregation window from 500ms to 200ms will increase dispatch frequency and improve effective throughput."
My response: I tested this hypothesis before accepting it. The result:

Configuration	Window	Steady Rate	vs Target
Baseline	500ms	988.4 ev/s	-1.2%
Test (window=200ms)	200ms	988.1 ev/s	-1.2%

Why I disagreed: The suggestion implied the pipeline was falling behind on dispatch frequency — but the p95 latency was only 6.55ms. If the pipeline were behind on processing, latency would be elevated. The 200ms window test confirmed this was not the bottleneck. The 0.3 ev/s difference (988.4 → 988.1) was noise, not signal.

Why this mattered: Chasing a false bottleneck would have wasted a full test cycle (6+ minutes) and potentially led to unnecessary production code changes. More importantly, the failed hypothesis redirected investigation toward the actual root cause — token bucket timing overhead in the load test harness, not the pipeline.
Lesson: When a tuning suggestion has no measurable effect, believe the data, not the theory.

Moment 2: Rejecting Batch Size Increase (200 events)

AI's suggestion: "Larger batches (200 events instead of 100) reduce per-request HTTP overhead and will help reach 1000 ev/s."
My response: I tested this independently, expecting it to fail based on the consistent ~988 ev/s pattern across all configurations. The result:

Configuration	Batch	Steady Rate	vs Target
Baseline	100	988.4 ev/s	-1.2%
Test (batch=200)	200	992.8 ev/s	-0.7%

Why I disagreed: The improvement from 988.4 to 992.8 ev/s (+4.4 ev/s) was real but insufficient to cross the 1000 ev/s threshold. More critically, I recognized that even at batch=200, the steady rate was still below target, which meant the bottleneck was not being addressed. If HTTP overhead were the root cause, batch=200 should have shown a more dramatic improvement.

The deeper concern: Increasing batch size to 200 in production would change the event batching semantics — each batch would represent 200 sensor events that might be delayed from appearing in the real-time chart. This is a product behavior trade-off, not just a performance tuning.

Why this mattered: The batch=200 test confirmed that HTTP overhead was a contributing factor but not the primary bottleneck. The remaining ~7 ev/s gap (from 992.8 to 1000) indicated there was still something else causing the shortfall — which led to the discovery of the token bucket timing issue.
Lesson: A test that partially improves but doesn't fully solve the problem is more informative than a test that fully solves it — it tells you where to look next.

Moment 3: Rejecting Flush Interval Reduction (50ms)

AI's suggestion: "Reducing EventFlush__IntervalMs from 1000ms to 50ms will increase SignalR dispatch frequency, reducing batching overhead and improving throughput."

My response: I treated this as a Phase 1 investigation, not a guaranteed fix. The result:

Configuration	Flush	Steady Rate	vs Target
Baseline	1000ms	988.4 ev/s	-1.2%
Test (flush=50ms)	50ms	989.3 ev/s	-1.1%

Why I disagreed: The 0.9 ev/s improvement was within normal variation. More importantly, the Flush worker already takes a snapshot every 50ms and dispatches to SignalR — but SignalR is fire-and-forget and the bottleneck was not in the dispatch path. The latency data (p95 6.55ms) already proved the pipeline was processing events quickly.

The critical question I asked: "If the pipeline is processing events in 3ms median time and dispatching to SignalR asynchronously, why would faster flushing matter?"

Answer: It doesn't. The flush interval determines how often we poll for completed aggregation windows. But since TumblingWindowAggregator fires a callback on window rollover, the flush timer is mostly redundant — it's a safety net, not the primary dispatch mechanism. Reducing it from 1000ms to 50ms just made the safety net run 20x more often without providing 20x more value.

Why this mattered: Changing `EventFlush__IntervalMs` in production would increase CPU usage on the pipeline with zero benefit to throughput or latency. This would have been a pure waste of resources.

Lesson: When a configuration parameter has a "obvious" improvement story but no measurable effect in testing, the "obvious" story is wrong. Trust the evidence.

AI's Proposed Solution Would Have Caused Performance Issues

The pattern across AI suggestions: Every pipeline configuration change (window size, batch size, flush interval) was based on the implicit assumption that **the bottleneck was in the pipeline**. All five suggestions aimed to tune pipeline internals:

1. Reduce aggregation window → faster dispatch
2. Increase batch size → fewer HTTP requests
3. Reduce flush interval → more frequent SignalR sends
4. Increase buffer capacity → fewer events dropped (but drop rate was already 0%)
5. Switch to async dispatch → "non-blocking" pipeline (but dispatch was already fire-and-forget)

What would have happened if I accepted AI's initial framing:
If I had accepted the assumption that "pipeline tuning is the answer," I would have continued making production code changes indefinitely, each time testing and finding the same ~988 ev/s ceiling. The iterative pipeline tuning would have:

1. **Changed `TumblingWindowAggregator` window size** — no effect, would have required redeployment

2. **Increased batch size to 200** — marginal improvement (992.8 ev/s) but still failing, and changes real-time chart semantics
3. **Reduced flush interval to 50ms** — marginal improvement (989.3 ev/s) but increases CPU usage
4. **Increased buffer capacity to 50,000** — no effect (drop rate was already 0%, buffer was never full)
5. **Switched to synchronous SignalR dispatch** — would have **worsened** performance by blocking the dispatch thread

The worst-case scenario: If AI had suggested switching SignalR dispatch from fire-and-forget to synchronous wait-for-confirmation, that would have transformed the pipeline from a ~3ms per-event processing time to a blocking I/O wait on every aggregation cycle — likely dropping throughput by 50% or more.

The correct insight (which AI eventually reached via systematic debugging): The pipeline is not the bottleneck. The ~1.2% shortfall is in the load generator's ability to sustain precise timing. The fix is a load test configuration change (`--target-rate 1020`), not a production code change.

Summary: Productive Tension in AI Collaboration

AI Suggestion	My Action	Result	Why I Was Right
Reduce window to 200ms	Tested first	988.1 ev/s (no change)	Pipeline wasn't behind — latency proved it
Increase batch to 200	Tested first	992.8 ev/s (marginal)	Root cause was load generator, not HTTP overhead
Reduce flush to 50ms	Tested first	989.3 ev/s (no change)	Flush is safety net, not primary dispatch path
(All pipeline tuning)	All tested	Same ~988 ev/s ceiling	Pipeline confirmed not the bottleneck

The principle that emerged: When every possible pipeline tuning yields the same result (~988 ev/s), the bottleneck is not in the pipeline — it's in the layer that measures the pipeline. In this case, the load test token bucket.

AI's role: Systematic debugging methodology (Phase 1: gather evidence, Phase 2: find patterns, Phase 3: form hypothesis, Phase 4: test minimally) provided the framework to prove my instincts correct.

Human's role: Questioning assumptions, demanding evidence before acceptance, and recognizing when "no effect" is itself meaningful data.